

Local RAG Project Assessment

Prepared for Aarushi Gupta | 19 September 2026

1 Project purpose and architecture

Verdict: This is a substantial, feature-rich document assistant with a credible foundation for an MSME pilot. It is not yet a validated production platform or a general-purpose business assistant. Its strongest work is document retrieval, source attribution, per-user access and defensive answer checks. The immediate investment should be reliable business answers and safe operations, not more model training.

What has actually been built. The main product is a custom React and TypeScript web application backed by FastAPI, PostgreSQL with pgvector, and Ollama. RAG means retrieving relevant company information before asking an existing language model to answer. Uploading documents changes the searchable knowledge base, not the model's parameters. The original Open WebUI proposal is now a separate deployment kit in server-build/; it is not the frontend of this application.

How a question travels: Browser with personal key -> API authentication and document permissions -> query embedding and database retrieval -> ranked evidence -> local model -> citation validation -> answer, sources and saved conversation. A bounded queue serializes model work to reduce contention. A permission-aware answer cache can reuse previous results when the question, settings and corpus version match.

Component	Responsibility and implementation
Browser and API	frontend/src/ provides the interface; app.py serves the API and built frontend on port 8000. Devices display results rather than run the model.
Model runtime	Ollama normally listens on loopback 11434. An optional separate embedding instance uses 11435. Model profiles trade output length and retrieval depth against cost.
Knowledge and state	PostgreSQL stores extracted text, chunks, vectors, table rows, permissions, chats, jobs, caches and evaluation results. pgvector searches meaning; full-text search matches words.
Optional helpers	PaddleOCR extracts scanned text. A separate cross-encoder service on loopback 8091 can rerank results. BGE-M3 has a non-serving experimental index.
Deployment	PowerShell scripts cover startup, network access, import scheduling, database migration and backup. The separate Open WebUI kit uses port 8080 and its own service setup.

Configuration inspected today. The workspace .env selects gemma3:1b-it-qat and embeddinggemma, requires an API key, enables strict citation validation, and disables automatic migrations and automatic model routing. No cross-encoder endpoint is configured. The database address is remote, not loopback; therefore, this configuration is not wholly local even if inference is. Extracted company text and chat records are stored at that database destination. The office-hours resource policy is also disabled. These are file settings, not verification of a running process or its environment overrides; no credentials are reproduced here.

Scope of this assessment. The review covered backend, frontend, retrieval, ingestion, authentication, evaluation, training preparation, operational scripts and deployment documentation at commit 9bb53ab, including existing uncommitted training-preparation files. Offline checks were run; live databases, services, model weights and Tally were not changed. Current production uptime, installed models and server throughput are not established by this audit.

Source anchors: app.py:56 configuration, app.py:991 schema, app.py:4126 streaming; frontend/src/App.tsx; scripts/start-local-rag.ps1; server-build/README.md.

2 How the implemented product works

User experience and access. Users can chat, select speed/quality profiles, scope questions to a document, upload files, inspect citations, stop generation, retry or edit the last question, and search, rename, export or delete conversations. The interface includes light/dark themes, mobile layouts, ingestion progress and system metrics. Admin tools create users, rotate keys and grant document access. The backend also supports key expiry, model restrictions, request limits and account deactivation, although not every control has a complete UI. Authentication uses personal API keys, not a conventional password/SSO login. This is user-level access control inside one installation, not company-level multi-tenant isolation.

Knowledge ingestion. Uploads support PDF, DOCX, XLSX, CSV, JSON, text/Markdown and common images. The pipeline extracts text or invokes OCR, forms smaller searchable chunks with larger parent context, embeds them, then saves content and metadata. Checksums detect duplicate files; background jobs expose progress. Table rows are also stored as structured records. Lifecycle fields can mark active, archived or superseded documents. However, DOCX page labels are synthetic, spreadsheet labels refer to sheets, and original uploads are not retained as a complete document archive. OCR confidence is not consistently preserved by the helper. These limits matter when users need to verify an invoice or cite an exact page.

Retrieval and answer construction. The backend combines vector search, keyword search and identifier matching, merges rankings, removes repetitive evidence and adds parent or adjacent context. Optional cross-encoder reranking is a separate enhancement, not the current configured path. It packs selected evidence into a server-controlled prompt, asks for citations and refuses unsupported answers. Client-supplied system messages are not retained as privileged instructions. Citation checks add a useful defensive layer, but use ordered word matching and English negation heuristics, not a proof that every claim logically follows. Correct paraphrases and translations can be rejected; some unsupported interpretations can still pass.

Important boundaries. Only a small set of greetings and conversational phrases bypass retrieval. Most other questions require document evidence, so open-ended writing, coding and general reasoning are not first-class modes. Summaries use a bounded set of chunks, not necessarily the entire document. “Total” and aggregation queries retrieve a limited selection of rows; they do not execute a complete, permission-scoped financial calculation. A stock balance, revenue total or debtor ageing report must not be treated as authoritative from this path. Keyword tokenization is ASCII/English-oriented, which weakens Marathi and Hindi retrieval even when embeddings understand those languages.

Integration and training status. Drive ingestion is a download-and-index workflow using shareable links, not a private OAuth connector that inherits Drive permissions and mirrors deletions. The /v1/chat/completions route resembles an OpenAI chat endpoint but expects X-API-Key; clients using only Bearer authentication are not drop-in compatible. No Tally accounting connector or transaction execution workflow is implemented. Training work consists of review preparation, approved-conversation export and configuration, not demonstrated fine-tuning. The September 7 preparation report records 4,422 source-review candidates and zero approved training examples; it is not evidence of a trained model.

What “generalized” should mean here. Build one reusable application with three explicit modes: company knowledge with citations; general assistance clearly labelled as not company-verified; and business analytics through controlled data tools. Keep each company's changing facts and access rules in retrieval, not shared model weights. Industry packs should change vocabulary, templates, connectors and evaluation examples without forking the core application.

Source anchors: document_processing.py; drive_sync.py; app.py:1988 routing, app.py:2240 keyword query, app.py:2577 grounding; rag_core/grounding.py; training/MCCIA_DATA_REVIEW.md.

3 Engineering quality and release readiness

Evidence checked on 19 September. All 91 offline tests passed across test_rag, test_upgrade and the training-review tests. The frontend passed a no-output TypeScript check. These are meaningful regression signals, but they do not establish real model accuracy, browser usability, database permissions, restore success or behaviour under multiple users. The evaluation file contains 21 questions; none satisfy the project's human-review requirements. Its coverage gate requires at least 150 reviewed cases across 16 categories. Consequently, there is no defensible current answer-accuracy percentage.

What is good. The code goes beyond a basic chatbot wrapper: source tracking, permission-filtered retrieval, per-user limits, corpus-aware caching, refusal checks, ingestion status, cancellation and evaluation tooling are implemented. Tests target several earlier grounding and cache defects. Migration tools avoid automatic cutover, restrict local targets and provide verification/restore workflows. These are useful engineering foundations worth retaining.

Performance is not yet a service guarantee. The September 5 baseline, on an approximately 7.33 GiB laptop, reported hybrid retrieval median/p95 of 3.714/4.674 seconds. Answer medians were 4.692-8.030 seconds across available profiles, but generation used only one question repeated three times per profile. Available RAM fell to 0.015 GiB. A later comparison run was stopped by the memory guard. These are historical, narrow measurements, not current Xeon results. Strict grounded-answer validation buffers the answer before displaying it, so an open streaming connection does not mean immediate token-by-token text. Cache hits also retain original generation timings, making some displayed latency metrics misleading.

Material finding	Business consequence and release requirement
Privacy and authentication	The inspected database is remote. Default code can permit an anonymous administrator when both key requirements and the master key are absent; today's file configuration enables authentication. Fail closed, verify actual bindings and test access isolation before any wider rollout.
Tally protection and availability	The resource policy is disabled in the inspected file and, when enabled, applies during office hours rather than imposing hard process limits. Ingestion/warmup are not covered by the same admission check. Queue limits are process-local; startup is at user logon, not an unattended boot service.
Data integrity and operations	Replacement deletes the old document before new embeddings finish, so a failure can remove the usable version. The least-privilege SQL grants only four legacy tables, omitting current users, permissions and other tables. Require atomic replacement and a tested runtime role/restore drill.
Correctness and maintainability	Small evidence windows, incomplete aggregation, English-oriented matching and character-based context budgeting constrain quality. A roughly 4,600-line backend and 1,200-line UI concentrate change risk. Legacy CLI defaults/schema assumptions diverge from the app; no repository CI workflow is present.

Assessment: Feature coverage is strong for a prototype; production assurance is incomplete. Single-company, supervised document Q&A is the best initial use. Automated accounting decisions, cross-company hosting and unattended production co-hosting with Tally should wait. Missing PATCH in cross-origin settings is also a concrete integration defect for separately hosted frontends. Central security audit history, retention controls, end-to-end tests and measured recovery procedures remain important gaps. A passing unit suite must not be presented as a production approval.

Source anchors: test_rag.py; test_upgrade.py; rag_core/evaluation.py; reports/baseline_before_upgrade.md; app.py:386 auth, app.py:590 resource gate, app.py:1513 replacement, app.py:4154 cache metrics; sql/neon_least_privilege.sql.

4 Improvements for a reusable MSME assistant

Priority 0 - make the existing product trustworthy. First complete a supervised, single-company pilot: fail-closed authentication; verified HTTPS/private access; an explicit local-versus-hosted database decision; a complete least-privilege role; atomic document replacement; and backup/restore with measured recovery. Separate AI from Tally where feasible, or use a separately validated resource budget covering chat, ingestion, OCR and warmup. Do not lower the memory guard to make a demo pass. Fix latency accounting and cross-origin methods, reconcile deployment instructions, and add CI plus real database/API/browser tests. Split the backend into authentication, ingestion, retrieval, generation and persistence modules without changing external behaviour.

Priority 1 - deliver business outcomes, not just chat. Start with three workflows whose answers staff can independently verify. The following are proposed additions, not existing capabilities.

MSME use case	Implementation direction and acceptance evidence
Policies, SOPs and product knowledge	Department-scoped collections, document owner/effective date, explicit supersession and evidence links. Test conflicting and expired policies; deny cross-role access.
Accounts and inventory questions	A read-only Tally export/import or supported connector, reviewed schemas and deterministic SQL/calculations. Show company, date range, filters, last-sync time and reconciled totals. Never infer totals from top-ranked chunks.
Sales and customer support	Product/pricing lookup, quotation and email drafts using approved templates. Require human review before sending or changing records; test prices against the source catalogue.

Priority 1 - generalization and regional language quality. Add explicit Company Knowledge, General Assistant and Business Analytics modes with visible boundaries. Ask clarification when the company, period or document is ambiguous. Make search Unicode-aware, handle Marathi/Hindi and mixed-language business terms, and review bilingual answers with language-competent staff. Use token-aware budgeting and hierarchical summaries for long documents. Evaluate the existing embedding/reranker alternatives on held-out company questions before promotion; a larger model or a more forceful prompt is not evidence of improvement.

Priority 2 - safe growth. Before serving several MSMEs, introduce a company/tenant identifier throughout documents, users, jobs, retrieval and caches, with database-enforced isolation and adversarial access tests. Add company onboarding, roles, quotas, private connectors with permission/deletion synchronization, retention and deletion workflows, and an administrator audit trail. Invoice OCR should expose the original image, confidence and a correction queue before posting data. Voice input, WhatsApp-style channels and approval-based actions can follow; they should not precede accuracy and isolation. Fine-tuning is optional for a measured behaviour problem, not a substitute for current business data or access controls.

Release gates. Build the existing minimum 150-question human-reviewed set, then extend it with representative MSME, multilingual, numeric and permission tests. Agree accuracy and refusal thresholds with business owners before testing. Require zero observed unauthorized disclosures in the acceptance suite, reconciliation of every test accounting total, and successful restoration into a disposable environment. On the actual deployment machine, measure cold/warm/cache-hit first-visible-answer and total p50/p95 latency for one and three users while observing Tally's own response time. Set the latency target from that pilot, not from the old laptop benchmark.

Bottom line: Preserve the current retrieval and access-control foundation. Position the next release as a reliable company assistant with transparent evidence and read-only business tools. That is a practical route to a generalized MSME product; claiming a universally capable or newly trained LLM now would overstate what the repository demonstrates.

Source anchors: scripts/verify-postgres-restore.ps1; scripts/verify-migration.py; scripts/evaluate-reviewed.py; scripts/evaluate-shadow-index.py; server-build/50-tally-load-test.ps1. Priorities and release gates above are recommendations from this audit, not completed work.